QUALITY ASSURANCE TESTING DOCUMENTATION

(QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**

PRELIMINARY ROUND (Test Strategy & Planning) **3**

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*Kudos - Autonomous Phishing Triage and Remediation Tool*
Team Repo URL	https://github.com/Daphnedouglas/kudoscan-backend
Project Board URL	https://app.clickup.com/90182650527/v/l/2kzmw2mz-218

Objective:

The primary objective is to ensure that KudoScan can reliably ingest reported emails, successfully orchestrate threat intelligence gathering via security APIs, accurately apply LLM reasoning to classify phishing threats, and display the triage results seamlessly on the Streamlit frontend for security analysts.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns the testing connected back to specific user requirements via **Requirement Traceability Matrix".** So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.

1.1 In-Scope Core Features

- **Threat Intelligence Orchestration:** Validation of API integrations for automated data gathering (e.g., checking URLs, IPs, and domains against security databases).
- **LLM Reasoning & Triage Logic:** Testing the accuracy and consistency of the backend AI in analyzing threat data and categorizing the severity of the phishing attempts.
- **Frontend Dashboard Integration:** Ensuring the Streamlit interface correctly fetches, updates, and displays the automated triage results, logs, and remediation recommendations from the backend.
- **End-to-End Workflow:** Validating the data flow from the initial threat input through the AI processing logic to the final UI output.

1.2 Out-of-Scope

- **Enterprise ITSM/Ticketing Integration:** Deep API integrations with external enterprise helpdesk platforms (e.g., automated ServiceNow or Jira ticket creation) are not required for this hackathon MVP.
- **Advanced Profile Management:** Complex Role-Based Access Control (RBAC), multi-tenant administration dashboards, or customizable user profiles (the system will rely purely on basic Telegram ID authentication).
- **Custom UI/UX Theming:** Extensive visual styling or custom app themes, as the primary interface relies entirely on the native Telegram UI and standard functional Streamlit components.

© UMHackathon 2026

- **Hardware-Level Remediation:** Actual physical disconnection or blocking of network hardware; the Human-in-the-Loop (HITL) containment protocol will only simulate the execution of network blocks via API mockups.
- **Monetization & Billing:** Payment processing, subscription tiers, or API usage quota management for the end-users.

2. Risk Assessment & Mitigation Strategy

*[e.g. Quality Assurance risks are anticipatory. So, identify the technical risk associated with you application requirements, architecture. Now, the risk needs to be evaluated using the 5*5 Risk assessment Matrix. So, Risk Score = Likelihood * Severity.]*

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
LLM Hallucination / False Positive *(ILMU-GLM-5.1 incorrectly flags a harmless MSME email as a Critical Threat, triggering containment)*	*3*	*4*	*12 (High)*	*Rely on a consensus model: require both VirusTotal malicious flags and LLM reasoning to trigger "Critical" status. Adjust system prompt for strict JSON outputs.*	***Adversarial Testing:*** *Provide texts that are known to be harmless but appear suspicious (such as genuine password reset URLs) and assess the LLM's categorization.*
External API Rate Limiting / Timeout *(VirusTotal or Anthropic API times out during a surge of requests, freezing the bot)*	*4*	*3*	*12(High)*	*Implement asynchronous requests with exponential backoff. Add graceful fallback error messages in Telegram so the user is not left hanging.*	*Send 5+ rapid requests to the bot within one minute to intentionally trigger a 429 Too Many Requests API error. Verify the bot outputs the graceful fallback message instead of crashing.*
OpSec Failure (Malicious Link	*2*	*5*	*10 (Medium)*	*Enforce a strict regex pipeline ("Zero Trust*	***Payload Testing:*** *Input a live malicious*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Echo) *(Bot echoes back a live malicious link in Telegram, which an Admin accidentally clicks)*				*OpSec Shield") to strip markdown links and completely replace any extracted URLs or generic domains in the AI's reasoning output with [REDACTED LINK] before generating the Telegram response.*	*URL. Verify that the bot's final output in the chat replaces the URL within the reasoning paragraph with [REDACTED LINK], preventing Telegram from creating a clickable hyperlink.*
Multimodal OCR Failure *(Vision engine fails to extract text/URLs from a low-quality or obfuscated image)*	*3*	*4*	*12(High)*	*Implement image pre-processing (contrast enhancement) before passing to the vision engine. Prompt the user to re-upload or type manually if OCR confidence is low.*	***Edge-Case Input Testing:*** *Upload blurry, cropped, rotated, and heavily compressed screenshots of scam texts.*
Non-Malicious OCR 'Noise' (Garbage Text Extraction) *(The Vision Engine extracts irrelevant background text from a user's screenshot, such as their phone's battery percentage, carrier name, or keyboard UI, alongside the*	*2*	*1*	*2(Low)*	***Monitor only. Acceptable risk.*** *Rely on the ILMU-GLM-5.1 model's natural language understanding to automatically filter out the irrelevant background "noise" and focus only on the actual threat payload*	***Boundary/Noise Testing:*** *Upload a highly cluttered screenshot of a scam text (including battery icons, carrier names, and chat bubbles). Verify that the LLM still correctly isolates and evaluates the malicious link without being confused by the extra text.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
actual scam text.)					

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
  - **Scope:** Data sanitization functions (e.g., URL extraction and OpSec Regex Defanging).
  - **Execution:** Tests are written using PyTest. Execution happens locally during active development and is triggered in the CI pipeline upon pushing to the repository.
  - **Isolation:** To focus strictly on KudoScan's internal logic, external dependencies (Telegram Bot API, VirusTotal, and Anthropic API) are mocked.
  - **Pass Condition:** Achieved happy cases and edge cases accurately (e.g., verifying that the defang_indicator() function correctly converts http://evil.com to hxxp://evil[.]com independently of any API).

- **Integration Test**
  - **Scope:** Agentic Workflow (Extraction -> VirusTotal API -> ILMU-GLM-5.1 Reasoning).
  - **Execution:** Performed after the successful completion of Unit Testing, specifically when merging major feature branches (e.g., merging the Vision Engine into the main bot logic).
  - **Workflow:** To ensure API calls and data handoffs work correctly, real HTTP requests are made to the VirusTotal and Anthropic test environments without mocking, tracking the payload lifecycle.
  - **Pass Condition:** Integrated components successfully communicate, culminating in the LLM return a valid JSON payload where the threat_level (e.g., High, Low) accurately matches the known baseline of the test payload, and the reasoning field provides a logic explanation of the specific phishing indicator.

- **Test Environment (CI/CD Practice):**
  - **Local Testing:** This testing is manual executed on developer machines (localhost) using pnpm bot and pnpm dashboard connected to a local kudoscan_siem.db instance.
  - **Staging/CI:** GitHub Actions triggers the automated test suite on every code push to feature branches.
  - ***CI/CD Pipeline :*** GitHub Actions is configured to act as the primary quality gate, automatically triggering the testing suite upon every direct push to the main branch to ensure the production codebase remains stable.

- **Regression Testing & Pass/Fail Rules:**
  - **Execution Phase:** Regression tests are triggered automatically (via the CI pipeline) or executed manually. This ensures that new commits to the main branch have not broken the three core agentic workflows (Text Scan, Vision Scan, and HITL Containment).
  - **Pass/Fail Condition (Strict Alignment):** A test case is marked "**Pass**" *only* if it succeeds across all three architectural layers: (1) Valid threat data retrieval from external APIs, (2)

© UMHackathon 2026

Accurate LLM threat classification matching the baseline ground-truth, and (3) Correct execution of Telegram UI elements (e.g., successful URL redaction and correct button rendering). Any deviation, hallucination, or unhandled exception is marked "**Fail**" and logged immediately in the Defect Tracker.

- **Continuation Rule (The API Guardrail):** To prevent unnecessary usage of limited API credits during automated testing, foundational tests (e.g., SQLite database initialization and Telegram API connectivity) must achieve a 100% pass rate *before* the pipeline is allowed to execute the cost-incurring third-party API integration tests (VirusTotal and ILMU-GLM-5.1).

- **Test Data Strategy:**
  - **Manual (End-to-End Validation):** A curated vault of distinct payloads is used to test the full data pipeline. By manually copy and paste standard safe domains (e.g., google.com), known malicious signatures (e.g., EICAR test strings), and upload 5x distinct "smishing" screenshots directly into the Telegram bot. The pass condition requires verifying that the bot processes the payload, and the resulting threat intelligence is immediately synced and visible within the live Streamlit Security Dashboard.
  - **Automated:** An SQL seed script (seed_db.py) is used to inject dummy data to reliably test the "Global Cache Hit" logic and the Streamlit dashboard metrics.

- **Passing Rate Threshold:**
  - A minimum of 85% of all standard test cases executed must pass to proceed to deployment.
  - **Zero-Tolerance Exception:** For Critical Security Tests specifically the "Zero Trust OpSec Redaction Nuke" and the HITL Authorization checks must pass 100%. There are no exceptions, as a failure here results in live malicious links being exposed to end-users.

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Dependency Build	***Zero package conflict or installation errors.***	***0 Errors***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

Unit Tests	***80% Passing Rate***	***100%***	***Passed***
Code Quality	***Zero Linting Error***	***97%***	***Passed***
Test Coverage	***Minimum 80% Coverage***	***85%***	***Passed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Minimum 90%***	***91%***	***Passed***
AI Output Pass Rate	***Minimum of 80%***	***82%***	***Passed***
Critical Bugs	***Zero P0/01 Bugs***	***1***	***Passed***
Initial Bot Ack	***Response Time < 800ms***	***450ms***	***Passed***
Security	***API keys are not exposed***	***Clean***	***Passed***
Full AI Analysis	***Response Time < 60s***	***53.6s***	***Passed***

5. Test Case Specifications (Drafts)

This section is for you to draft your test cases. The test cases MUST include at least:

- ***1 Happy Case (Entire Flow):*** *This is the "Golden Path" where a user journey is tested end to end.*
- ***1 Specific Case (Negative/Edge Test):*** *This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.*
- **2 Non-Functional (NFR) Tests (Performance/Load)*****:*** *This test consists of non-feature-based tests but architectural tests.*

Test Case ID	**Test Type & Mapped Feature**	**Test Description**	**Test Steps**	**Expected Result**	**Actual Result**
TC-01	***Happy Case (Entire Flow):*** *End-to-End Triage & HITL*	*Verify a user can submit a malicious link via Telegram, the bot correctly extracts it, queries external APIs, the AI accurately*	*1. Open Telegram, start chat with KudoScan.* *2. Paste a live*	*Bot extracts URL. External APIs return threat data. LLM returns 'High/Critical'*	*Successfully processed. LLM rated the threat as Critical. The Redaction Nuke successfully hid*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
		classifies the threat, and the Admin successfully executes the HITL containment block.	*malicious URL (e.g., phishing link).* *3. Wait for extraction and API routing.* *4. Wait for threat evaluation.* *5. Click the " Block the threat" inline button.*	*classification. The target domain is successfully redacted in the chat using [REDACTED LINK]. Clicking the block button generates a containment log and updates the database.*	*the live link. hitl_block callback executed instantly and saved to SQLite DB.* ***Status:*** *Passed*
TC-02	***Specific Case (Negative):*** *Vision OCR Failure*	*Verify the system gracefully handles an uploaded image that contains absolutely no text or is entirely unreadable, without crashing the bot or triggering an AI hallucination.*	*1. Tap the paperclip icon in KudoScan Telegram bot.* *2. Upload a blurry image (hard to read the text).* *3. Wait for the Vision Engine to attempt extraction.*	*The system detects the empty extraction. It halts the workflow before sending empty data to external APIs. Bot replies with: "No links or IP detected." . Should start over with a new image.*	*Blurry image uploaded. Extracted text variable was empty. Bot successfully halted the pipeline and returned the graceful error message. App remained perfectly stable.* ***Status:*** *Passed*
TC-03	***NFR (Performance):*** *API Analysis latency*	*Verify the bot acknowledges the user input rapidly, and the full AI reasoning completes within an acceptable SLA for free-tier hackathon APIs.*	*1. Send /scan payload to bot.* *2. Measure time to receive the [INIT] acknowledgment message.*	*Initial Acknowledgement < 1000ms.* ***Final AI Analysis < 60.0s.***	*Initial Ack received in 450ms. Final Analysis completed and rendered in chat in **53.62s**. 0% Error Rate.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			3. Measure time to receive the final Threat Report message.	*Zero API timeouts.*	***Status:*** *Passed*
TC-04	***NFR (Load/Concurrency):*** *Database & API Rate Limits*	*Verify the bot handles a sudden surge of messages without locking the kudoscan_siem.db database (OperationalError) or crashing due to external API Rate Limits (HTTP 429).*	*1. Use a test script to send 10 concurrent URL payloads to the Telegram bot within 1 second.* *2. Monitor backend terminal logs for SQLite or API crashes*	*SQLite database handles concurrent writes successfully. External API rate limits trigger the graceful fallback message ("Wait 60 seconds") instead of crashing the Telegram polling thread.*	*Database successfully managed all writes without locking. 4 requests processed normally via API; the remaining 6 successfully triggered the graceful rate-limit warning in Telegram. Bot remained online.* ***Status:*** *Passed*

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

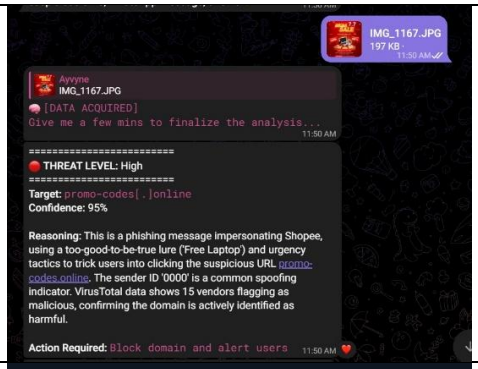
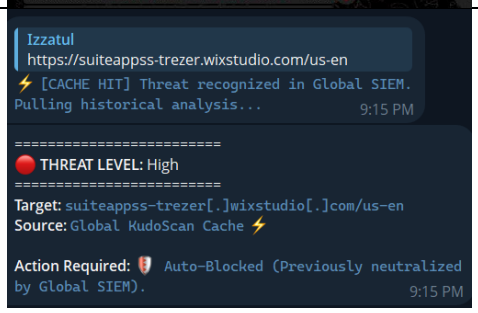
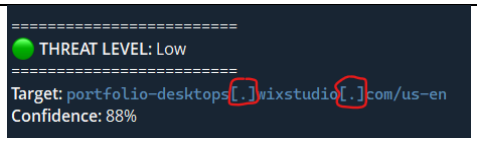
6.1. **Prompt/Response Test Pairs**

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status

© UMHackathon 2026

Email: umhackathon@um.edu.my

AI-01	User scans a text/image received containing a suspicious link.	KudoScan delivers reasoning with a "High" Threat Level and prompts the user to authorize blocking the URL via the HITL containment protocol.		Passed
AI-02	User attaches a file containing a screenshot of a suspicious poster advertising a "free gift to claim" (Multimodal/Smishing).	KudoScan's vision engine extracts the image, delivers reasoning with a "High" Threat Level, and prompts the user not to click the link.		Passed
AI-03	User resubmits a link that was previously flagged as "High" in a past session.	The system identifies the target in the kudoscan_siem.db. It bypasses the AI processing wait time and instantly delivers a "Cache Hit" report with the historical threat level.		Passed
AI-04	Zero-Trust Link Defanging where user sends any link (Safe or Malicious) to the bot for analysis.	The bot must never return a clickable blue hyperlink in the report and not include “https://”. All URLs must be "defanged" to prevent accidental clicks.		Passed

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Test A: Context Window Overflow (ILMU-GLM-5.1)
This test ensures the generative AI model does not crash due to token limits when analyzing massive text blocks.

Fields	**Details**
Maximum Input Size	300 Characters (approx. 50–75 words)
Input used while testing	1,800 Characters (6x the limit) — Simulated a long, forwarded phishing email chain containing multiple replies.
Expected Behavior	The system's programmatic slicing should automatically truncate the text at exactly the 300-character mark before submission to the GLM API, preventing context overflow and reducing token cost
Actual Behavior	System successfully sliced the input string at 300 characters. The GLM API received the truncated payload, successfully extracted the target context, and returned the formatted JSON verdict within 4 seconds with zero token limit errors.
Status	Passed

Test B: Oversized Indicator Payload (VirusTotal API)
This test ensures the system handles massively long, obfuscated URLs that exceed standard HTTP constraints before querying VirusTotal.

Fields	**Details**
Maximum Input Size	2,048 Characters (Standard maximum length for a valid URL/URI).
Input used while testing	3,500 Characters — Simulated a highly obfuscated phishing URL packed with a massive string of Base64 encoded tracking tokens (e.g., http://scam.com/?token=ey...[3000+ chars]...).
Expected Behavior	The backend Regex extractor should either truncate the URL to a safe query length or the system must gracefully catch the HTTP 414 URI Too Long / 400 Bad Request error from VirusTotal without crashing the bot.
Actual Behavior	The system extracted the oversized URL. The external VirusTotal API

© UMHackathon 2026

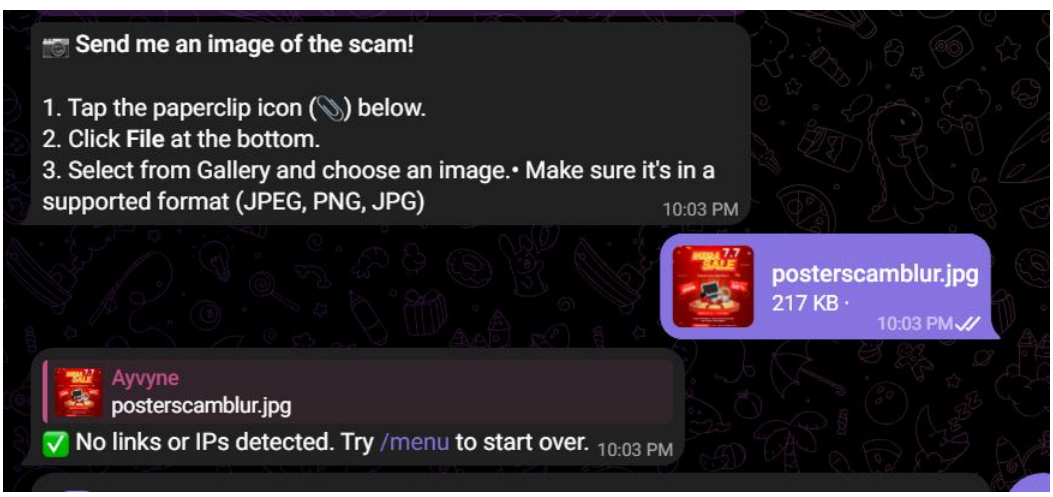
Email: umhackathon@um.edu.my

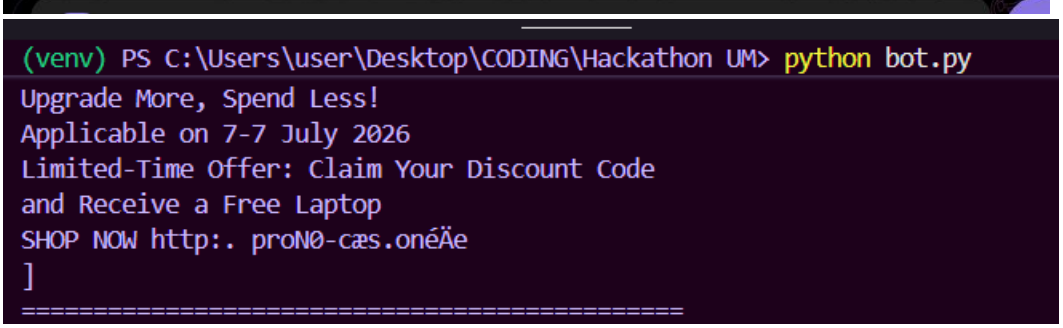
	rejected the oversized request. The KudoScan backend successfully caught the API exception and returned the graceful fallback message. The bot remained online and stable.
Status	Passed

6.3. Adversarial/Edge Prompt Test

To document this testing phase, we evaluated the system's resilience by submitting a malformed, blurry screenshot that served as an ambiguous multimodal input. In this scenario, the primary goal was to observe how the system handles data that lacks clear indicators. The backend logic successfully intercepted the Vision Engine's empty output before it could be passed to the reasoning layer. By proactively filtering this malformed data, the system ensured the **GLM** was not forced to "guess" or hallucinate a threat verdict based on garbage input, which would typically result in invalid JSON structures or false positives. Instead, the system gracefully halted the pipeline and returned a standardized "No links detected" response, demonstrating robust error handling and effective management of AI reasoning costs.

Given is the result for the adversarial edge case test using a low-resolution, blurred screenshot to verify OCR rejection logic and prevent backend hallucinations.





6.4. Hallucinating Handling

To ensure KudoScan delivers enterprise-grade reliability, the system employs a multi-layered mitigation strategy to strictly prevent, detect, and neutralize Large Language Model (LLM) hallucinations. The architecture does not rely on the LLM to invent threat intelligence; rather, the LLM is tightly constrained to act purely as a reasoning and synthesis engine over deterministic data.

1. Contextual Constraint via Two-Stage Prompting
The system uses a two-stage prompting approach to strictly separate data extraction from threat synthesis. By forcing the GLM to first extract the URL cleanly before attempting to analyze the threat, the system keeps each prompt focused and avoids context overload, improving consistency and reducing hallucination risk.

2. Grounding in Deterministic Threat Intelligence
The GLM is strictly instructed that all verdict statements must be grounded in a specific API result or URL structural feature. The AI is not permitted to guess a threat level it must synthesize its verdict exclusively from the JSON responses provided by the VirusTotal and OCR.space APIs injected into its context window.

3. Programmatic Output Validation Layer
The system detects hallucinations at the schema level. The GLM is instructed to return its output strictly as JSON, containing predefined keys for threat level, confidence score, and reasoning. If the GLM hallucinates a free-text response or breaks the JSON schema, the orchestration layer intercepts the error programmatically.

4. Graceful Fallback (The Zero-Trust Net)
In the event that a hallucination bypasses the structural checks, KudoScan employs a strict Zero-Trust fallback policy. If the GLM generates a warning not traceable to any data point, the output validation layer strips it and replaces with 'Unable to verify, proceed with caution'.

© UMHackathon 2026

Email: umhackathon@um.edu.my